D222 + LoRMA

Phase 4 Court Judgment Prediction | source-audited documentation

Problem statement and data

Phase 4 Court Judgment Prediction predicts binary outcome 0/1 from legal case text. The preserved official inputs are Train/train.json, Train/dev.json and Test/test.json. Test labels are private. Original run records report 994 official dev labels and 1,517 test IDs; macro-F1 is the central metric.

Architecture and mechanics

The 24-layer D222 causal decoder supplies contextual token states. Multiplicative low-rank updates adapt selected weights, while the evidence head learns token salience and ordered case-level context. Data flow: Legal case -> D222 own tokenizer/backbone -> Rank-8 multiplicative low-rank adaptation -> Token evidence pooling -> Ordered chunk BiGRU -> 2-class logits -> prediction.json.

Scientific rationale

D222: 24-layer causal decoder with grouped-query attention and alternating local/global context This experiment isolates the effect of its recorded adaptation or document-head mechanism while retaining the original task and export contract. Comparisons across different backbones, training budgets or data coverage remain descriptive rather than controlled causal estimates.

Exact run configuration and resource tradeoff

Experiment family d222; adaptation method lorma; original parent checkpoint SHA-256 08a737d002a71a90ad21f350e25a76f1e70eb172badf0671ff8818d96ea5c46f. Configuration: 256 tokens/chunk, 8 chunks/case, 8192 source characters considered, maximum 12000 training cases, 256 holdout cases per class. Optimizer configuration: backbone learning rate 0.0002, head learning rate 0.0003, gradient accumulation 8, max epochs 4, planned training hours 2.5. Recorded 213 optimizer updates and 1704 seen training examples; selected step 213. Selected checkpoint SHA-256 002e0dedacc5e8283ad8519c2ee6e6c9e3af15c83e3bf4cc2ee61178499b151a; parameters 226171274, trainable 4091402. Original run completed after nominal deadline: True.

Exact prediction path

For each unpadded chunk, the original backbone yields contextual token states. The head layer-normalizes them, then makes a softmax-weighted mixture of token mean, learned token attention and the last token. Normalized chunk positions enter through a learned projection. A bidirectional GRU contextualizes chunks in document order; learned attention and a chunk mean are mixed, then LayerNorm/dropout and a two-logit classifier predict the binary judgment.

Evaluation mechanics

Official labeled dev cases 994; class supports [497, 497]; confusion matrix rows true 0/1, columns predicted 0/1: [[95, 402], [92, 405]]. At selected threshold 0.284564, macro-F1 0.449472; at threshold 0.5, macro-F1 0.333333. The report keeps these thresholds separate because threshold choice changes the binary predictions. The submitted ZIP has 1,517 unique test IDs and binary labels; no test labels were available to this run.

Adaptation mechanism

LoRMA multiplies selected original linear transforms by trained low-rank factors (rank 8, alpha 16 here); the unchanged parent checkpoint remains the initialization.

Training design and checkpoint selection

Rank-8 multiplicative low-rank adaptation. Executed v3 runner: code/v3_t4x2.ipynb, code/runner/phase4_cjpe/kaggle_runner.py with published_v3_schedule.json; the six jobs share EvidenceCaseModel in code/runner/phase4_cjpe/models.py.

Training objective and validation boundary

The runner uses AdamW with separate backbone/head learning rates and weight decay 0.01. For each training example it computes class-weighted cross-entropy; weights are capped at 3.0. It periodically evaluates the fixed train-only holdout, saves the best checkpoint, and exports predictions from selected weights. A fixed, balanced training-only holdout of up to 512 cases selected the checkpoint and decision threshold. The official 994-case labeled dev set was evaluated after that selection; selected-threshold dev macro-F1 was 0.449472, while default 0.5 gave 0.333333. The hidden test labels were not available. These dev metrics are not the Codabench score shown in the user screenshot.

Measured results

Original Kaggle run: complete; 213 optimizer updates; selected step 213. Official 994-case dev macro-F1 0.449472 at recorded threshold 0.284564; default 0.5 dev macro-F1 0.333333. Verified prediction ZIP SHA-256 3be58f83f018fb99b1e491e0b69ff4507581b005de2f19d5fc52b8d01aa5af28, exactly 1,517 IDs. User-provided Codabench screenshot: 0.4650 (finished submission).

Source and evidence trail

Initial six-job Kaggle v3 runner; the per-model notebook in code/ is prepared source, while v3_t4x2.ipynb and archived Kaggle outputs identify the executed run. The code SHA-256 manifest, original run records, selected-checkpoint identity, and verified prediction archive are indexed in evidence/README.md. This report is a later reconstruction of preserved evidence, not a claim that its companion notebook ran the original training.

Prediction contract and verification

The original prediction ZIP contains one root-level prediction.json with 1,517 unique official test IDs and integer labels restricted to 0 and 1. This package checks the ZIP CRC and SHA-256 against the archived run/verification record. Full model checkpoint tensors remain outside GitHub; their downloaded backup receipts and hashes live in the local Phase 4 study archives.

Reproducibility and limits

The local dev score uses a threshold recorded in the run; inspect its calibration/selection record before treating it as independent model-selection evidence. A prepared model notebook is source scaffolding, while v3_t4x2.ipynb and the Kaggle run evidence identify the executed v3 lineage. Reports and companion notebook were created after training; original scores come from copied artifacts. Model weights and course data are preserved locally, not uploaded to GitHub.